

# AI Assisted coding

N.Jashwanth reddy

2303A53017

Batch-46

---

## Task 1: Automated Vulnerability Scanning

**Objective:** Use AI to simulate a Static Analysis Security Testing (SAST) tool.

- **Prompt:** *"Act as a SAST tool. Scan the following Python code for OWASP Top 10 vulnerabilities and provide a severity rating."*
- **Code Provided:** A basic Flask app with `os.system(user_input)`.
- **AI Findings:** Identified **Command Injection** (High Severity).
- **Refactored Solution:**

Python

```
import subprocess
```

```
# Secure: Use a list of arguments and no shell=True
```

```
def run_ping_secure(host):
```

```
    # Validates input implicitly by not invoking a shell
```

```
    result = subprocess.run(["ping", "-c", "4", host], capture_output=True, text=True)
```

```
    return result.stdout
```

---

## Task 2: Implementing Content Security Policy (CSP)

**Objective:** Secure a web application against data injection.

- **Prompt:** *"Generate a secure HTTP header configuration for a web app to prevent XSS and Clickjacking."*
- **Solution (Middleware Implementation):**

Python

```
@app.after_request
```

```
def add_security_headers(response):
```

```
    response.headers['Content-Security-Policy'] = "default-src 'self';"
```

```
    response.headers['X-Frame-Options'] = 'DENY'
```

```
    response.headers['X-Content-Type-Options'] = 'nosniff'
```

```
    return response
```

---

### Task 3: Secure API Authentication (JWT)

**Objective:** Replace basic authentication with secure JSON Web Tokens.

- **Vulnerability:** Sending passwords in every request header.
  - **Refactored Solution:** \* **Prompt:** *"Refactor this basic auth login to use JWT with a 15-minute expiration."*
    - **Logic:** The AI implemented PyJWT with an exp claim and a secret key stored in an environment variable.
- 

### Task 4: Error Handling & Information Leakage

**Objective:** Ensure stack traces aren't exposed to end-users.

- **Insecure Code:** except Exception as e: return str(e)
- **Secure Refactoring:**

Python

```
import logging
```

```
logging.basicConfig(level=logging.INFO)
```

```
try:
```

```
    # Dangerous operation
```

```
val = 1 / 0
```

except Exception as e:

```
logging.error(f"Internal Error: {e}") # Log the real error for devs
```

```
return "An internal error occurred. Please try again later.", 500 # Safe message
```

Task 5: Comparison Audit (Summary Table)

| Security Control      | Manual Approach   | AI-Assisted Secure Refactoring               |
|-----------------------|-------------------|----------------------------------------------|
| Input Sanitization    | Basic strip()     | Complex Regex & Type enforcement             |
| Data Privacy          | Cleartext storage | Hashing with bcrypt / Argon2                 |
| Dependency Management | Manual updates    | Automated vulnerability patching suggestions |